

Sincronizacion

Algo muy piola que encuentre en quora:

When multiple threads need to work on the same data structure and multiple changes need to happen either all at once or not at all, then you need a lock. It is tempting to use spin locks, but they have two failure modes that cause trouble.

The first one is the same as other uses of atomics: congested atomics tend to be slow and use a lot of resources.

The second problem is that if the thread holding the lock is descheduled, then no one can make any progress until it runs again and releases the lock.

Las variables normales tienen mas cache hits que las atómicas **porque el TestAndSet sobrescribe la variable con true activando la invalidación de cache lines en todos los caches de procesadores donde la variable esta guardada** (Simplemente porque sobrescribe en vez de solo leer). (coherencia del cache, protocolo MESI, Orga2, Furfi (?)).

TAS: variable.TestAndSet(). Devuelve el valor de la variable y la pone en true de manera atómica. (Hace las dos cosas a la vez).

TTAS: variable.get() and variable.TestAndSet(). permite usar un while donde se hace variable.get() para obtener el valor *SIN SOBRESCRIBIR*. Si el valor es false entonces recién ahí hacemos el test and set atómico.

CAS: variable.CompareAndSwap(a, b). Permite comparar una variable con algún valor. Si el actual *variable* es igual a *a* entonces devuelve *true* y en *variable* pone *b*.

Caso contrario devuelve false.

Es parecido a TAS pero en vez de poner true en la variable compara la variable con otro valor y si son iguales cambia por b.

¿Para qué necesitamos sincronización entre procesos? ¿Qué soluciones nos ofrece el HW? Explicar el caso para monoprocesador y para multiprocesador. (instrucciones atómicas y deshabilitar interrupciones)

Los procesos manipulan varios recursos y en general puede ocurrir que dos procesos cambien el estado de un dato al mismo tiempo evitando que sus rutinas se completen de manera consistente.

El HW nos ofrece dos instrucciones que permite establecer atómicamente el valor de una variable entera en 1. **TestAndSet** y **CompareAndSwap**.

TestAndSet pone 1 y devuelve el valor anterior de forma atomica.

CompareAndSwap compara de manera atomica 2 valores y si los dos son iguales entonces modifica el contenido objetivo con un nuevo valor

Esto nos ayuda a modelar las secciones criticas. El problema es que hay que usar busy waiting.

```
while(TestAndSet(&lock)) {
    // Si estaba en true entonces sigue lockeado. Quiero que diga false
    para cortarme el while
}

// Seccion Critica

lock = false;
```

Otra solucion que nos ofrece el hardware son *Mutex*. La implementacion consta de que lock() usa busy waiting (como el codigo de arriba) esperando que la seccion critica este disponible y luego entra. Tambien se lo conoce como *spin-lock*.

| Estar esperando en un mutex no previene interrupciones.

Monoprocesador Cuando un proceso ejecuta wait() o signal() se deshabilitan las interrupciones de otros procesos.

Esto implica que dos procesos no pueden ejecutar wait() o signal() de un mismo semaforo al mismo tiempo. Solo ejecuta el proceso actual hasta que las interrupciones se habiliten nuevamente y el scheduler pueda reobtener el control.

Multiprocesador: Se deshabilitan las interrupciones para todos los procesadores. Si no, instrucciones de diferentes procesos pueden intervenir de manera arbitraria en nuestros recursos.

¿Cómo nos afecta si el scheduler es preemptive o non-preemptive en la implementación de un semáforo?

```
typedef struct {
    int value;
    struct process *list;
} semaphore;

void wait(semaphore *S) {
```

```

S->value--;
if (S->value < 0) {
    add this process to S->list;
    sleep();
}
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}

```

Preventivo: Puede ocurrir que cuando estamos ejecutando un wait() o un signal() se desaloje el proceso a la mitad de la ejecución de estas funciones y otro proceso modifique la misma data permitiendo que dos procesos entren a la sección crítica.

Necesitamos que wait() y signal() sean atómicas

Voluntario/No-Preventivo: Hay más seguridad a la hora de ejecutar wait() o signal().

Evaluar si están bien o mal utilizadas en los siguientes ejemplos las primitivas de sincronización:

- Usar TASLock (spinlock) para acceder a disco.
 - Usar semáforos para incrementar un contador.
 - Usar un contador atómico para un recurso que tienen que poder acceder 3 procesos a la vez.
 - usar spinlock para un recurso que tienen que poder acceder 3 procesos a la vez.
1. No vale la pena porque mientras se busca la data del disco el proceso está en busy waiting usando rafagas de CPU para nada. Se podría suspender al proceso (mandarlo a la cola de procesos en waiting) y que otros procesos tengan más rafaga por unidad de tiempo.
 2. Sería más conveniente usar una variable atómica e incrementarla de manera atómica en un entorno altamente concurrente.
 3. En este caso lo mejor es un semaforo. Si usamos un semaforo podemos establecer que solo 3 procesos puedan tomar la zona crítica a la vez y los otros procesos

esperando que se envíen a la waiting queue. Si usáramos un contador atómico necesitaríamos busy waiting consumiendo así demasiados recursos.

4. Nuevamente un semáforo. El spinlock usa busy waiting y además solo bloquea a 1 proceso, no a 3.

Ojo. Los casos 3 y 4 dependen del poder de cómputo que consume la zona crítica. Si la zona crítica es muy chiquita puede ser posible que el consumo del busy waiting sea menor al overhead de enviar procesos a la Waiting Queue y luego ponerlos en la Ready Queue. En estos casos es preferible usar variables atómicas.

Diferencia entre spin lock y semáforos (hablar de TTAS). ¿En qué contexto preferimos uno sobre el otro y por qué?

El spin lock bloquea el acceso a la zona crítica de un solo proceso usando variables atómicas con busy waiting usando TTASLock donde el while hace cache hits en cada iteración (si se ejecuta en el mismo procesador, tal vez afinidad dura) mientras que los semáforos, si usamos la implementación a nivel sistema operativo, cambian el estado de los procesos de Running a Waiting cuando ejecutan `sem.wait()` con el contador del semáforo en 0 además de que los semáforos soportan múltiples threads para entrar en la zona crítica.

Los spin locks están buenos para cuando queremos que un solo proceso entre a la zona crítica donde la zona crítica es "chica". Si la zona crítica es grande puede que el busy waiting consuma más CPU que el overhead de dormir y despertar otros procesos que quieren entrar a esta.

Los semáforos son útiles cuando queremos que múltiples procesos entren a una zona crítica al mismo tiempo, la capacidad inicial se establece en la creación del semáforo. Si un proceso entra a la zona crítica se le resta 1 a la capacidad actual del semáforo. Si un proceso sale aumenta en 1 la capacidad del semáforo (idealmente).

Esta capacidad siempre se mantiene capeada porque si entran X procesos luego salen exactamente X entonces nunca pueden entrar X y salir X+1.

¿Cómo implementamos una sección crítica con spinlocks?

1. Inicializamos una **variable atómica** (por ejemplo, `bool lock = false;`) que indica si un proceso está dentro de la zona crítica. En su estado inicial (false), ningún proceso está dentro de la zona crítica.
2. Antes de entrar a la sección crítica, usamos un **bucle while** que verifica si la variable es **true** (indica que la zona crítica está ocupada).
 - Si la variable es **true**, el proceso vuelve a chequear (gira) esperando.

- Si la variable es **false**, el proceso sale del bucle y pone la variable en **true** para indicar que está entrando a la zona crítica.
3. Es importante que esta operación de "leer y escribir" en la variable sea **atómica**, ya que si no lo fuera, podría producirse un **cambio de contexto** (context switch) entre la lectura y la escritura. Esto podría hacer que otro proceso entre a la zona crítica antes de que el primer proceso marque la variable como ocupada.
- Para hacer esto atómicamente, se utiliza una instrucción como **TestAndSet**, que primero verifica si la variable está en **false** (libre), y luego la establece en **true** (ocupada), todo en un solo paso atómico. Esto evita que dos procesos puedan acceder a la zona crítica simultáneamente.
4. **Optimización:** Se puede reducir el overhead utilizando una variante de **TestAndSet** conocida como **TestAndTestAndSet**. Este patrón consiste en hacer primero una comprobación no atómica del valor de la variable, y solo si es **false** (indica que la zona crítica está libre), se realiza la operación atómica **TestAndSet**. Esto evita realizar operaciones atómicas innecesarias, lo cual mejora el rendimiento.

Explicar el problema clásico de lectores y escritores. Explicar cómo podemos evitar inanición del escritor.

Sucede cuando varios procesos quieren escribir y leer al mismo tiempo un mismo recurso. Como una base de datos.

Lectores: Solo leen el recurso. Múltiples lectores pueden acceder al recurso simultáneamente. No hay ningún problema.

Escritores: Modifican el recurso. Solo un escritor puede modificar el recurso a la vez y no pueden existir lectores concurrentes *durante* la escritura.

El objetivo es maximizar la concurrencia y mantener la consistencia de los datos.

1. Exclusion mutua para escritores: Solo un escritor puede tener acceso para modificar el recurso a la vez.
2. Acceso concurrente para lectores: Si no hay escritores, todos los lectores pueden leer el recurso simultáneamente.
3. Evitar inanición: Todos los procesos pueden escribir y leer el recurso en un tiempo razonable.

Inanición del escritor: Muchos procesos entran repetidamente al recurso para leer evitando que el escritor pueda escribir, demorándolo indefinidamente.

Inanición del lector: Muchos procesos bloquean para su escritura particular demorando la lectura indefinidamente.

La solucion mas justa es una FIFO indiferente de lectores o escritores.